

Développement et Analyse des Performances d'une IA pour un Wargame Tactique

Zakaria Saidane

M1-IISC

1. Introduction et Règles du Jeu

L'objectif de ce TP est de concevoir et d'optimiser une Intelligence Artificielle autonome capable de jouer et de gagner à un jeu de wargame au tour par tour. Le jeu se déroule sur une grille à deux dimensions de taille 10×10 (soit 100 cases, indexées de la ligne 0 à 9 et de la colonne A à J). Deux joueurs s'affrontent : les Blancs (notés `x`), qui commencent la partie en bas du plateau (lignes 0, 1 et 2), et les Noirs (notés `o`), placés en haut (lignes 7, 8 et 9).

Chaque joueur dispose d'un ensemble d'unités dont les valeurs numériques varient (1, 2 ou 3), représentant la puissance brute de chaque pièce. À chaque tour de jeu, un joueur peut déplacer l'une de ses unités d'une seule case dans les 8 directions possibles (horizontale, verticale et diagonale). Les contraintes de déplacement sont les suivantes :

1. Il est interdit de sortir des limites physiques du plateau 10×10.
2. Il est interdit de déplacer une pièce sur une case déjà occupée par l'une de ses propres unités (pas de tir fratricide).
3. Si la case de destination est occupée par une pièce ennemie, un affrontement (bataille) est déclenché.

La résolution d'une bataille repose sur le calcul de la somme des valeurs des pièces présentes dans le voisinage direct des 8 cases entourant la case de destination (x, y). L'attaquant capture la pièce adverse si et seulement si la somme des pièces de sa couleur dans les 8 cases voisines est strictement supérieure à la somme des pièces ennemies situées dans ce même voisinage. Le perdant de la bataille voit son unité éliminée du plateau.

Une partie se termine par la victoire d'un joueur sous deux conditions explicites :

- Soit l'un des joueurs a réussi à éliminer la totalité des pièces adverses du plateau.
- Soit l'un des joueurs parvient à amener l'une de ses pièces sur la ligne de fond adverse (ligne 9 pour les Blancs, ligne 0 pour les Noirs).

2. Algorithme Décisionnel : Minimax et Élagage Alpha-Beta

2.1 Exploration Arborescente et Fonctions `f_min` / `f_max`

Pour permettre à l'IA de planifier ses coups, nous avons mis en œuvre l'algorithme Minimax. Cet algorithme d'exploration en profondeur parcourt l'arbre des possibilités en alternant deux points de vue :

- La fonction `f_max`, qui simule le tour de notre IA visant à maximiser le score heuristique.
- La fonction `f_min`, qui simule le tour de l'adversaire cherchant à minimiser le score de notre IA.

Conformément aux directives du sujet, la profondeur de recherche est fixée à 3 demi-coups ($d = 3$). Cela signifie que l'algorithme anticipe trois coups à l'avance avant d'évaluer la situation du jeu.

Dans notre implémentation, la signature des fonctions `f_min` et `f_max` a été modifiée par rapport au squelette de base en ajoutant le plateau en paramètre (`Pion* jeu`). Cela permet de simuler des déplacements virtuels sur des copies temporaires du plateau (obtenues via `f_raz_plateau` et `f_copie_plateau`), sans

jamais altérer l'état réel de la partie en cours. L'exploration récursive s'arrête lorsqu'une feuille est atteinte, ce qui correspond à l'une des trois conditions suivantes : l'atteinte de la profondeur maximale (0), une situation de victoire/défaite immédiate (`f_gagnant != 0`), ou le déclenchement d'une coupure Alpha-Beta.

2.2 Implémentation des Coupures Alpha-Beta

Afin d'éviter d'évaluer des branches inutilement chronophages, nous avons intégré l'élagage Alpha-Beta. Deux variables (`alpha` et `beta`) sont propagées dans l'arbre :

- `alpha` conserve la meilleure valeur trouvée jusqu'ici pour le joueur MAX.
- `beta` conserve la meilleure valeur trouvée jusqu'ici pour le joueur MIN.

Dans `f_min`, si la valeur calculée devient inférieure ou égale à `alpha`, l'exploration de cette branche est immédiatement interrompue car le joueur MAX a déjà un meilleur choix garanti ailleurs. Inversément, dans `f_max`, si la valeur dépasse ou égale `beta`, une coupure est effectuée.

Afin d'évaluer l'impact réel de cet élagage (comme demandé lors des travaux pratiques), nous avons ajouté une variable globale `g_use_alpha_beta` qui sert d'interrupteur. Lorsque cet interrupteur est désactivé, l'algorithme parcourt l'intégralité de l'arbre décisionnel (Minimax naïf), ce qui nous donne un point de comparaison quantitatif direct.

2.3 Ordonnancement des Coups (Move Ordering)

L'efficacité des coupures Alpha-Beta dépend fortement de l'ordre dans lequel les coups sont explorés dans chaque nœud. Si l'algorithme teste en premier les coups les plus prometteurs, la valeur d'alpha ou de beta est ajustée très vite, ce qui provoque des coupures bien plus tôt dans la recherche.

Pour cela, nous avons écrit une fonction `f_get_order_score` qui attribue un score rapide à chaque coup légal avant la récursion :

1. **Victoire immédiate** (score de 1000) : si le mouvement permet d'atteindre la ligne de fond adverse.
2. **Captures réussies** (score de 500) : si l'attaque d'un pion ennemi se solde par une victoire lors de la bataille.
3. **Attaques simples** (score de 100) : si le coup engage un combat sans garantie de victoire.
4. **Avancement** (score de 20 + distance) : favorise la progression vers le camp adverse.
5. **Contrôle du centre** (score de 5) : encourage l'occupation des cases centrales.

Les coups générés à chaque nœud sont ensuite triés par ordre décroissant de score à l'aide de la fonction C standard `qsort`et de notre comparateur `compareMoves`. Ainsi, les coups jugés « intelligents » sont analysés en premier.

3. Élaboration de la Fonction d'Évaluation Heuristique (`f_eval`)

La fonction `f_eval` représente la logique stratégique de notre IA. Elle prend en paramètre l'état du plateau ainsi que le joueur courant, et renvoie un score numérique indiquant à quel point la position est favorable.

Un score très élevé indique un avantage net pour le joueur, tandis qu'un score négatif indique une position défavorable.

Pour élaborer une heuristique performante, nous avons combiné plusieurs critères selon une hiérarchie stricte de coefficients :

3.1 Hiérarchie et Table des Coefficients

1. **Victoire / Défaite (Priorité Absolue - Coef: $\pm 1\,000\,000$)** : Si l'état du jeu indique une victoire définitive (pion sur la ligne de fond ou plus d'ennemis), la fonction renvoie une valeur infinie ($1\,000\,000$). C'est le critère qui surpasse tous les autres.
2. **Proximité de la Victoire (Bonus: $+1000$)** : Lorsqu'un pion atteint l'avant-dernière ligne (ligne 1 pour les Noirs ou ligne 8 pour les Blancs), un bonus massif de 1000 points est accordé. Cela incitera fortement l'IA à faire le dernier pas au tour suivant pour clore la partie.
3. **Valeur Matérielle (Coef: 15)** : Calculée par la fonction `f_valeur`, elle représente la somme des valeurs des pièces restantes (1, 2 ou 3). Le coefficient de 15 garantit qu'une pièce de valeur 3 a plus de poids que deux pièces de valeur 1, préservant ainsi nos unités puissantes.
4. **Avancement Géographique (Coef: 10 en temps normal / 30 en fin de partie)** : Récompense la progression des pièces vers la ligne de fond adverse. Nous avons implémenté une **stratégie adaptative** : lorsque le nombre de pièces ennemies passe sous le seuil de 5 et que l'IA conserve au moins 5 pièces (mode « poussée finale »), le facteur d'avancement est triplé (de 10 à 30). De plus, les pions faibles (valeur 1) reçoivent un bonus supplémentaire s'ils s'infiltrent en profondeur.
5. **Nombre de Pièces (Quantité - Coef: 5)** : Calculé par `f_nbPions`, ce critère valorise la quantité d'unités possédées, ce qui offre une flexibilité tactique sur le terrain.
6. **Contrôle du Centre (Coef: 5)** : Accorde un petit bonus pour chaque pièce occupant la zone centrale du plateau (lignes 3 à 6, colonnes C à G). Cela permet d'éviter que les pièces restent bloquées sur les bords en début de partie.

3.2 Récapitulatif des Pondérations :

- **Victoire / Défaite** : $\pm 1\,000\,000$
- **Proximité Victoire** : $+1000$
- **Valeur Matérielle** : 15
- **Avancement (Normal)** : 10
- **Avancement (Fin de partie)** : 30
- **Nombre de pièces** : 5
- **Contrôle du centre** : 5

4. Processus de Développement et Réglages Expérimentaux

La mise au point de cette fonction d'évaluation a nécessité un long travail d'itération et de nombreuses parties de test pour corriger des comportements indésirables :

1. **Première Version (Stagnation et Répétition)** : Lors des premiers essais avec Minimax, l'IA restait très passive et répétait des mouvements aller-retour (gauche-droite) sans chercher à attaquer ni avancer. Tous les coups d'ouverture se valaient selon la fonction d'évaluation de base. Pour casser

cette monotonie, nous avons ajouté un léger bruit aléatoire (entre -25 et $+25$) lors des 5 premiers tours de jeu. Cela a permis de varier les ouvertures sans dégrader la qualité globale des décisions.

2. **Deuxième Version (Problème d'Embouteillage Central)** : Après avoir ajouté le critère de contrôle du centre, nous avons constaté que l'IA avait tendance à envoyer l'ensemble de ses pièces vers le milieu du plateau. Cela créait un blocage massif où nos propres unités se gênaient, permettant à l'adversaire de contourner nos forces par les côtés pour atteindre la ligne de fond. Nous avons donc réduit le poids du contrôle du centre à 5 et augmenté la priorité des attaques et de l'avancement.
3. **Troisième Version (Passivité et Parties Interminables)** : En augmentant l'agressivité pour favoriser les captures, l'IA est devenue extrêmement défensive : elle cherchait uniquement à bloquer et capturer les pièces adverse qui approchaient, sans jamais prendre l'initiative d'avancer vers la ligne ennemie. Les parties IA vs IA ne se terminaient plus.
4. **Version Finale (Stratégie Adaptative et Move Ordering)** : Pour résoudre ce problème, nous avons instauré le mode « poussée finale » (multiplicateur $\times 3$ sur l'avancement dès qu'un avantage numérique est détecté) et le bonus de proximité de victoire ($+1000$). Enfin, l'intégration de l'ordonnancement des coups (`Move Ordering`) a accéléré le temps de calcul en permettant aux coupures Alpha-Beta de s'effectuer beaucoup plus vite.

5. Analyse des Résultats et Comparaison Alpha-Beta

Grâce au système de statistiques intégré dans notre programme (`g_stats`), nous avons mesuré précisément les performances de l'algorithme sur une série de parties complètes à profondeur 3.

5.1 Résultats Expérimentaux (Parties IA vs IA à Profondeur 3) :

```
--- Statistiques Minimax (avec Alpha-Beta) ---
Profondeur de recherche : 3
Nombre de tours d'IA joués : 86
Total des appels f_min/f_max : 990 251
Total des noeuds (fils) analysés : 985 642
Total des coupures Alpha-Beta : 239 296
Temps total d'exécution IA : 1.3152 secondes
Temps moyen par tour : 0.0153 secondes
Moyenne des coupures par tour : 2782.51

--- Statistiques Minimax (sans Alpha-Beta) ---
Profondeur de recherche : 3
Nombre de tours d'IA joués : 36
Total des appels f_min/f_max : 8 153 760
Total des noeuds (fils) analysés : 8 151 607
Total des coupures Alpha-Beta : 0
Temps total d'exécution IA : 9.2842 secondes
Temps moyen par tour : 0.2579 secondes
Moyenne des coupures par tour : 0.00
```

5.2 Interprétation des Résultats :

- **Réduction du Nombre de Nœuds** : L'activation de l'élagage Alpha-Beta permet de passer de plus de 8,1 millions de nœuds analysés à moins de 986 000 nœuds pour un nombre de tours comparable. Cela représente une réduction de près de 88 % de l'arbre d'exploration.

- **Gain en Temps de Calcul** : Le temps moyen d'exécution par tour passe de 0.2579 seconde (sans Alpha-Beta) à seulement 0.0153 seconde (avec Alpha-Beta). L'IA est ainsi environ **17 fois plus rapide** à répondre tout en prenant des décisions strictement identiques sur le plateau.
- **Efficacité du Move Ordering** : Avec une moyenne de plus de 2 780 coupures par tour, l'ordonnancement des coups montre son efficacité : les branches inintéressantes sont rapidement coupées dès la vérification des premiers fils.

6. Optimisations Bonus et Choix d'Implémentation

Afin d'assurer la stabilité et les performances du programme lors du tournoi, plusieurs choix techniques et optimisations ont été réalisés :

- **Gestion Propre de la Mémoire** : Dans les fonctions de recherche, les allocations de plateaux temporaires sont systématiquement suivies d'appels à `free()` pour éviter toute fuite mémoire au cours des parties de longue durée.
- **Mesure Précise du Temps** : Nous avons utilisé la fonction `gettimeofday` (en microsecondes) pour obtenir des statistiques de temps fiables.
- **Variabilité des Ouvertures** : Le léger bruit stochastique (± 25) appliqué sur le score des 5 premiers tours empêche l'IA d'être totalement prévisible lors de confrontations répétées contre la même IA.
- **Compilation et Optimisation GCC** : Le code a été testé et compilé avec les flags d'optimisation `-O1`, `-O2` et `-O3` de GCC, produisant des exécutables très réactifs pour le tournoi.

7. Conclusion

Ce TP nous a permis de comprendre concrètement le fonctionnement de l'algorithme Minimax et de mesurer l'impact déterminant des coupures Alpha-Beta associées à un bon ordonnancement des coups. Nous avons appris que le réglage d'une fonction d'évaluation heuristique est un processus empirique délicat : accorder un poids trop important à un seul critère crée immédiatement un comportement obsessionnel de l'IA (comme l'embouteillage au centre ou un jeu exclusivement défensif). La hiérarchie des objectifs doit s'adapter à la phase de la partie (ouverture, milieu et fin de jeu). En conclusion, l'IA développée propose un jeu équilibré : elle défend ses pièces de valeur, effectue des attaques opportunes, contrôle le plateau et bascule efficacement en mode poussée finale pour l'emporter. Pour aller plus loin, nous pourrions envisager d'ajouter des tables de transposition (hashing de Zobrist) afin de mémoriser les positions déjà évaluées, ou d'implémenter une profondeur de recherche dynamique en fonction du temps disponible.